

UNIVERSAL SOURCE CODING

Here we develop the basics of universal source coding. Minimax regret data compression is defined, and the descriptive cost of universality is shown to be the information radius of the relative entropy ball containing all the source distributions. The minimax theorem shows this radius to be the channel capacity for the associated channel given by the source distribution. Arithmetic coding enables the use of a source distribution that is learned on the fly. Finally, individual sequence compression is defined and achieved by a succession of Lempel–Ziv parsing algorithms.

In Chapter 5 we introduced the problem of finding the shortest representation of a source, and showed that the entropy is the fundamental lower limit on the expected length of any uniquely decodable representation. We also showed that if we know the probability distribution for the source, we can use the Huffman algorithm to construct the optimal (minimal expected length) code for that distribution.

For many practical situations, however, the probability distribution underlying the source may be unknown, and we cannot apply the methods of Chapter 5 directly. Instead, all we know is a class of distributions. One possible approach is to wait until we have seen all the data, estimate the distribution from the data, use this distribution to construct the best code, and then go back to the beginning and compress the data using this code. This two-pass procedure is used in some applications where there is a fairly small amount of data to be compressed. But there are many situations in which it is not feasible to make two passes over the data, and it is desirable to have a one-pass (or online) algorithm to compress the data that “learns” the probability distribution of the data and uses it to compress the incoming symbols. We show the existence of such algorithms that do well for any distribution within a class of distributions.

In yet other cases, there is no probability distribution underlying the data—all we are given is an individual sequence of outcomes. Examples

of such data sources include text and music. We can then ask the question: How well can we compress the sequence? If we do not put any restrictions on the class of algorithms, we get a meaningless answer—there always exists a function that compresses a particular sequence to one bit while leaving every other sequence uncompressed. This function is clearly “overfitted” to the data. However, if we compare our performance to that achievable by optimal word assignments with respect to Bernoulli distributions or k th-order Markov processes, we obtain more interesting answers that are in many ways analogous to the results for the probabilistic or average case analysis. The ultimate answer for compressibility for an individual sequence is the Kolmogorov complexity of the sequence, which we discuss in Chapter 14.

We begin the chapter by considering the problem of source coding as a game in which the coder chooses a code that attempts to minimize the average length of the representation and nature chooses a distribution on the source sequence. We show that this game has a value that is related to the capacity of a channel with rows of its transition matrix that are the possible distributions on the source sequence. We then consider algorithms for encoding the source sequence given a known or “estimated” distribution on the sequence. In particular, we describe arithmetic coding, which is an extension of the Shannon–Fano–Elias code of Section 5.9 that permits incremental encoding and decoding of sequences of source symbols.

We then describe two basic versions of the class of adaptive dictionary compression algorithms called Lempel–Ziv, based on the papers by Ziv and Lempel [603, 604]. We provide a proof of asymptotic optimality for these algorithms, showing that in the limit they achieve the entropy rate for any stationary ergodic source. In Chapter 16 we extend the notion of universality to investment in the stock market and describe online portfolio selection procedures that are analogous to the universal methods for data compression.

13.1 UNIVERSAL CODES AND CHANNEL CAPACITY

Assume that we have a random variable X drawn according to a distribution from the family $\{p_\theta\}$, where the parameter $\theta \in \{1, 2, \dots, m\}$ is unknown. We wish to find an efficient code for this source.

From the results of Chapter 5, if we know θ , we can construct a code with codeword lengths $l(x) = \log \frac{1}{p_\theta(x)}$, achieving an average codeword

length equal to the entropy $H_\theta(x) = -\sum_x p_\theta(x) \log p_\theta(x)$, and this is the best that we can do. For the purposes of this section, we will ignore the integer constraints on $l(x)$, knowing that applying the integer constraint will cost at most one bit in expected length. Thus,

$$\min_{l(x)} E_{p_\theta}[l(X)] = E_{p_\theta} \left[\log \frac{1}{p_\theta(X)} \right] = H(p_\theta). \quad (13.1)$$

What happens if we do not know the true distribution p_θ , yet wish to code as efficiently as possible? In this case, using a code with codeword lengths $l(x)$ and implied probability $q(x) = 2^{-l(x)}$, we define the redundancy of the code as the difference between the expected length of the code and the lower limit for the expected length:

$$R(p_\theta, q) = E_{p_\theta}[l(X)] - E_{p_\theta} \left[\log \frac{1}{p_\theta(X)} \right] \quad (13.2)$$

$$= \sum_x p_\theta(x) \left(l(x) - \log \frac{1}{p_\theta(x)} \right) \quad (13.3)$$

$$= \sum_x p_\theta(x) \left(\log \frac{1}{q(x)} - \log \frac{1}{p_\theta(x)} \right) \quad (13.4)$$

$$= \sum_x p_\theta(x) \log \frac{p_\theta(x)}{q(x)} \quad (13.5)$$

$$= D(p_\theta \| q), \quad (13.6)$$

where $q(x) = 2^{-l(x)}$ is the distribution that corresponds to the codeword lengths $l(x)$.

We wish to find a code that does well irrespective of the true distribution p_θ , and thus we define the *minimax redundancy* as

$$R^* = \min_q \max_{p_\theta} R(p_\theta, q) = \min_q \max_{p_\theta} D(p_\theta \| q). \quad (13.7)$$

This minimax redundancy is achieved by a distribution q that is at the “center” of the information ball containing the distributions p_θ , that is, the distribution q whose maximum distance from any of the distributions p_θ is minimized (Figure 13.1).

To find the distribution q that is as close as possible to all the possible p_θ in relative entropy, consider the following channel:

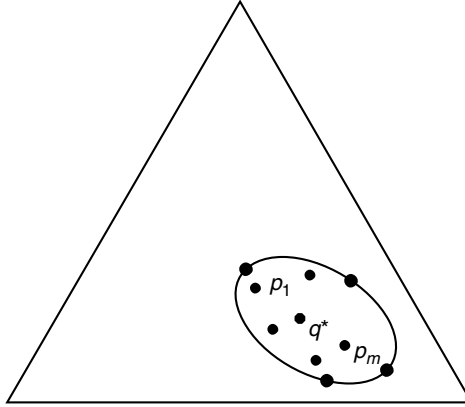


FIGURE 13.1. Minimum radius information ball containing all the p_θ 's

$$\theta \rightarrow \begin{bmatrix} \dots p_1 \dots \\ \dots p_2 \dots \\ \vdots \\ \dots p_\theta \dots \\ \vdots \\ \dots p_m \dots \end{bmatrix} \rightarrow X. \quad (13.8)$$

This is a channel $\{\theta, p_\theta(x), \mathcal{X}\}$ with the rows of the transition matrix equal to the different p_θ 's, the possible distributions of the source. We will show that the minimax redundancy R^* is equal to the capacity of this channel, and the corresponding optimal coding distribution is the output distribution of this channel induced by the capacity-achieving input distribution. The capacity of this channel is given by

$$C = \max_{\pi(\theta)} I(\theta; X) = \max_{\pi(\theta)} \sum_{\theta} \pi(\theta) p_\theta(x) \log \frac{p_\theta(x)}{q_\pi(x)}, \quad (13.9)$$

where

$$q_\pi(x) = \sum_{\theta} \pi(\theta) p_\theta(x). \quad (13.10)$$

The equivalence of R^* and C is expressed in the following theorem:

Theorem 13.1.1 (Gallager [229], Ryabko [450]) *The capacity of a channel $p(x|\theta)$ with rows $p_1, p_2, \dots, p_m$ is given by*

$$C = R^* = \min_q \max_{\theta} D(p_\theta \| q). \quad (13.11)$$

The distribution q that achieves the minimum in (13.11) is the output distribution $q^*(x)$ induced by the capacity-achieving input distribution $\pi^*(\theta)$:

$$q^*(x) = q_{\pi^*}(x) = \sum_{\theta} \pi^*(\theta) p_{\theta}(x). \quad (13.12)$$

Proof: Let $\pi(\theta)$ be an input distribution on $\theta \in \{1, 2, \dots, m\}$, and let the induced output distribution be q_{π} :

$$(q_{\pi})_j = \sum_{i=1}^m \pi_i p_{ij}, \quad (13.13)$$

where $p_{ij} = p_{\theta}(x)$ for $\theta = i$, $x = j$. Then for any distribution q on the output, we have

$$I_{\pi}(\theta; X) = \sum_{i,j} \pi_i p_{ij} \log \frac{p_{ij}}{(q_{\pi})_j} \quad (13.14)$$

$$= \sum_i \pi_i D(p_i \| q_{\pi}) \quad (13.15)$$

$$= \sum_{i,j} \pi_i p_{ij} \log \frac{p_{ij}}{q_j} \frac{q_j}{(q_{\pi})_j} \quad (13.16)$$

$$= \sum_{i,j} \pi_i p_{ij} \log \frac{p_{ij}}{q_j} + \sum_{i,j} \pi_i p_{ij} \log \frac{q_j}{(q_{\pi})_j} \quad (13.17)$$

$$= \sum_{i,j} \pi_i p_{ij} \log \frac{p_{ij}}{q_j} + \sum_j (q_{\pi})_j \log \frac{q_j}{(q_{\pi})_j} \quad (13.18)$$

$$= \sum_{i,j} \pi_i p_{ij} \log \frac{p_{ij}}{q_j} - D(q_{\pi} \| q) \quad (13.19)$$

$$= \sum_i \pi_i D(p_i \| q) - D(q_{\pi} \| q) \quad (13.20)$$

$$\leq \sum_i \pi_i D(p_i \| q) \quad (13.21)$$

for all q , with equality iff $q = q_{\pi}$. Thus, for all q ,

$$\sum_i \pi_i D(p_i \| q) \geq \sum_i \pi_i D(p_i \| q_{\pi}), \quad (13.22)$$

and therefore

$$I_{\pi}(\theta; X) = \min_q \sum_i \pi_i D(p_i \| q) \quad (13.23)$$

is achieved when $q = q_{\pi}$. Thus, the output distribution that minimizes the average distance to all the rows of the transition matrix is the the output distribution induced by the channel (Lemma 10.8.1).

The channel capacity can now be written as

$$C = \max_{\pi} I_{\pi}(\theta; X) \quad (13.24)$$

$$= \max_{\pi} \min_q \sum_i \pi_i D(p_i \| q). \quad (13.25)$$

We can now apply a fundamental theorem of game theory, which states that for a continuous function $f(x, y)$, $x \in \mathcal{X}$, $y \in \mathcal{Y}$, if $f(x, y)$ is convex in x and concave in y , and $\mathcal{X}, \mathcal{Y}$ are compact convex sets, then

$$\min_{x \in \mathcal{X}} \max_{y \in \mathcal{Y}} f(x, y) = \max_{y \in \mathcal{Y}} \min_{x \in \mathcal{X}} f(x, y). \quad (13.26)$$

The proof of this minimax theorem can be found in [305, 392].

By convexity of relative entropy (Theorem 2.7.2), $\sum_i \pi_i D(p_i \| q)$ is convex in q and concave in π , and therefore

$$C = \max_{\pi} \min_q \sum_i \pi_i D(p_i \| q) \quad (13.27)$$

$$= \min_q \max_{\pi} \sum_i \pi_i D(p_i \| q) \quad (13.28)$$

$$= \min_q \max_i D(p_i \| q), \quad (13.29)$$

where the last equality follows from the fact that the maximum is achieved by putting all the weight on the index i maximizing $D(p_i \| q)$ in (13.28). It also follows that $q^* = q_{\pi^*}$. This completes the proof. $\square$

Thus, the channel capacity of the channel from θ to X is the minimax expected redundancy in source coding.

Example 13.1.1 Consider the case when $\mathcal{X} = \{1, 2, 3\}$ and θ takes only two values, 1 and 2, and the corresponding distributions are $p_1 = (1 - \alpha, \alpha, 0)$ and $p_2 = (0, \alpha, 1 - \alpha)$. We would like to encode a sequence of symbols from $\mathcal{X}$ without knowing whether the distribution is p_1 or p_2 . The arguments above indicate that the worst-case optimal code uses the

codeword lengths corresponding to the distribution that has a minimal relative entropy distance from both distributions, in this case, the midpoint of the two distributions. Using this distribution, $q = \{\frac{1-\alpha}{2}, \alpha, \frac{1-\alpha}{2}\}$, we achieve a redundancy of

$$D(p_1 \| q) = D(p_2 \| q) = (1 - \alpha) \log \frac{1 - \alpha}{(1 - \alpha)/2} + \alpha \log \frac{\alpha}{\alpha} + 0 = 1 - \alpha. \quad (13.30)$$

The channel with transition matrix rows equal to p_1 and p_2 is equivalent to the erasure channel (Section 7.1.5), and the capacity of this channel can easily be calculated to be $(1 - \alpha)$, achieved with a uniform distribution on the inputs. The output distribution corresponding to the capacity-achieving input distribution is equal to $\{\frac{1-\alpha}{2}, \alpha, \frac{1-\alpha}{2}\}$ (i.e., the same as the distribution q above). Thus, if we don't know the distribution for this class of sources, we code using the distribution q rather than p_1 or p_2 , and incur an additional cost of $1 - \alpha$ bits per source symbol above the ideal entropy bound.

13.2 UNIVERSAL CODING FOR BINARY SEQUENCES

Now we consider an important special case of encoding a binary sequence $x^n \in \{0, 1\}^n$. We do not make any assumptions about the probability distribution for $x_1, x_2, \dots, x_n$.

We begin with bounds on the size of $\binom{n}{k}$, taken from Wozencraft and Reiffen [567] proved in Lemma 17.5.1: For $k \neq 0$ or n ,

$$\sqrt{\frac{n}{8k(n-k)}} \leq \binom{n}{k} 2^{-nH(k/n)} \leq \sqrt{\frac{n}{\pi k(n-k)}}. \quad (13.31)$$

We first describe an offline algorithm to describe the sequence; we count the number of 1's in the sequence, and after we have seen the entire sequence, we send a two-stage description of the sequence. The first stage is a count of the number of 1's in the sequence [i.e., $k = \sum_i x_i$ (using $\lceil \log(n+1) \rceil$ bits)], and the second stage is the index of this sequence among all sequences that have k 1's (using $\lceil \log \binom{n}{k} \rceil$ bits). This two-stage description requires total length

$$l(x^n) \leq \log(n+1) + \log \binom{n}{k} + 2 \quad (13.32)$$

$$\leq \log n + nH\left(\frac{k}{n}\right) - \frac{1}{2} \log n - \frac{1}{2} \log \left(\pi \frac{k(n-k)}{n}\right) + 3 \quad (13.33)$$

$$= nH\left(\frac{k}{n}\right) + \frac{1}{2} \log n - \frac{1}{2} \log \left(\pi \frac{k}{n} \frac{n-k}{n} \right) + 3. \quad (13.34)$$

Thus, the cost of describing the sequence is approximately $\frac{1}{2} \log n$ bits above the optimal cost with the Shannon code for a Bernoulli distribution corresponding to k/n . The last term is unbounded at $k = 0$ or $k = n$, so the bound is not useful for these cases (the actual description length is $\log(n+1)$ bits, whereas the entropy $H(k/n) = 0$ when $k = 0$ or $k = n$).

This counting approach requires the compressor to wait until he has seen the entire sequence. We now describe a different approach using a mixture distribution that achieves the same result on the fly. We choose the coding distribution $q(x_1, x_2, \dots, x_n) = 2^{-l(x_1, x_2, \dots, x_n)}$ to be a uniform mixture of all Bernoulli(θ) distributions on $x_1, x_2, \dots, x_n$. We will analyze the performance of a code using this distribution and show that such codes perform well for all input sequences.

We construct this distribution by assuming that θ , the parameter of the Bernoulli distribution is drawn according to a uniform distribution on $[0, 1]$. The probability of a sequence $x_1, x_2, \dots, x_n$ with k ones is $\theta^k(1 - \theta)^{n-k}$ under the Bernoulli(θ) distribution. Thus, the mixture probability of the sequence is

$$p(x_1, x_2, \dots, x_n) = \int_0^1 \theta^k (1 - \theta)^{n-k} d\theta \triangleq A(n, k). \quad (13.35)$$

Integrating by parts, setting $u = (1 - \theta)^{n-k}$ and $dv = \theta^k d\theta$, we have

$$\begin{aligned} \int_0^1 \theta^k (1 - \theta)^{n-k} d\theta &= \left[\frac{1}{k+1} \theta^{k+1} (1 - \theta)^{n-k} \right]_0^1 \\ &\quad + \frac{n-k}{k+1} \int_0^1 \theta^{k+1} (1 - \theta)^{n-k-1} d\theta, \end{aligned} \quad (13.36)$$

or

$$A(n, k) = \frac{n-k}{k+1} A(n, k+1). \quad (13.37)$$

Now $A(n, n) = \int_0^1 \theta^n d\theta = \frac{1}{n+1}$, and we can easily verify from the recursion that

$$p(x_1, x_2, \dots, x_n) = A(n, k) = \frac{1}{n+1} \frac{1}{\binom{n}{k}}. \quad (13.38)$$

The codeword length with respect to the mixture distribution is

$$\left\lceil \log \frac{1}{q(x^n)} \right\rceil \leq \log(n+1) + \log \binom{n}{k} + 1, \quad (13.39)$$

which is within one bit of the length of the two-stage description above. Thus, we have a similar bound on the codeword length

$$l(x_1, x_2, \dots, x_n) \leq H\left(\frac{k}{n}\right) + \frac{1}{2} \log n - \frac{1}{2} \log \left(\pi \frac{k}{n} \frac{(n-k)}{n} \right) + 2 \quad (13.40)$$

for all sequences $x_1, x_2, \dots, x_n$. This mixture distribution achieves a codeword length within $\frac{1}{2} \log n$ bits of the optimal code length $nH(k/n)$ that would be required if the source were really Bernoulli(k/n), without any assumptions about the distribution of the source.

This mixture distribution yields a nice expression for the conditional probability of the next symbol given the previous symbols of $x_1, x_2, \dots, x_n$. Let k_i be the number of 1's in the first i symbols of $x_1, x_2, \dots, x_n$. Using (13.38), we have

$$q(x_{i+1} = 1 | x^i) = \frac{q(x^i, 1)}{q(x^i)} \quad (13.41)$$

$$= \left(\frac{1}{i+2} \frac{1}{\binom{i+1}{k_{i+1}}} \right) / \left(\frac{1}{i+1} \frac{1}{\binom{i}{k_i}} \right) \quad (13.42)$$

$$= \frac{1}{i+2} \frac{(k_i+1)!(n-k_i)!}{(i+1)!} (i+1) \frac{k_i!(i-k_i)!}{i!} \quad (13.43)$$

$$= \frac{k_i+1}{i+2}. \quad (13.44)$$

This is the Bayesian posterior probability of 1 given the uniform prior on θ , and is called the *Laplace estimate* for the probability of the next symbol. We can use this posterior probability as the probability of the next symbol for arithmetic coding, and achieve the codeword length $\log \frac{1}{q(x^n)}$ in a sequential manner with finite-precision arithmetic. This is a horizon-free result, in that the procedure does not depend on the length of the sequence.

One issue with the uniform mixture approach or the two-stage approach is that the bound does not apply for $k=0$ or $k=n$. The only uniform bound that we can give on the extra redundancy is $\log n$, which we can obtain by using the bounds of (11.40). The problem is that

we are not assigning enough probability to sequences with $k = 0$ or $k = n$. If instead of using a uniform distribution on θ , we used the Dirichlet($\frac{1}{2}, \frac{1}{2}$) distribution, also called the Beta($\frac{1}{2}, \frac{1}{2}$) distribution, the probability of a sequence $x_1, x_2, \dots, x_n$ becomes

$$q_{\frac{1}{2}}(x^n) = \int_0^1 \theta^k (1 - \theta)^{n-k} \frac{1}{\pi \sqrt{\theta(1 - \theta)}} d\theta \quad (13.45)$$

and it can be shown that this achieves a description length

$$\log \frac{1}{q_{\frac{1}{2}}(x^n)} \leq H(k/n) + \frac{1}{2} \log n + \log \frac{\pi}{8} \quad (13.46)$$

for all $x^n \in \{0, 1\}^n$, achieving a uniform bound on the redundancy of the universal mixture code. As in the case of the uniform prior, we can calculate the conditional distribution of the next symbol, given the previous observations, as

$$q_{\frac{1}{2}}(x_{i+1} = 1 | x^i) = \frac{k_i + \frac{1}{2}}{i + 1}, \quad (13.47)$$

which can be used with arithmetic coding to provide an online algorithm to encode the sequence. We will analyze the performance of the mixture algorithm in greater detail when we analyze universal portfolios in Section 16.7.

13.3 ARITHMETIC CODING

The Huffman coding procedure described in Chapter 5 is optimal for encoding a random variable with a known distribution that has to be encoded symbol by symbol. However, due to the fact that the codeword lengths for a Huffman code were restricted to be integral, there could be a loss of up to 1 bit per symbol in coding efficiency. We could alleviate this loss by using blocks of input symbols—however, the complexity of this approach increases exponentially with block length. We now describe a method of encoding without this inefficiency. In arithmetic coding, instead of using a sequence of bits to represent a symbol, we represent it by a subinterval of the unit interval.

The code for a sequence of symbols is an interval whose length decreases as we add more symbols to the sequence. This property allows us to have a coding scheme that is incremental (the code for an extension to a sequence can be calculated simply from the code for the original sequence) and for which the codeword lengths are not restricted to be integral. The motivation

for arithmetic coding is based on Shannon–Fano–Elias coding (Section 5.9) and the following lemma:

Lemma 13.3.1 *Let Y be a random variable with continuous probability distribution function $F(y)$. Let $U = F(Y)$ (i.e., U is a function of Y defined by its distribution function). Then U is uniformly distributed on $[0, 1]$.*

Proof: Since $F(y) \in [0, 1]$, the range of U is $[0, 1]$. Also, for $u \in [0, 1]$,

$$F_U(u) = \Pr(U \leq u) \quad (13.48)$$

$$= \Pr(F(Y) \leq u) \quad (13.49)$$

$$= \Pr(Y \leq F^{-1}(u)) \quad (13.50)$$

$$= F(F^{-1}(u)) \quad (13.51)$$

$$= u, \quad (13.52)$$

which proves that U has a uniform distribution in $[0, 1]$. □

Now consider an infinite sequence of random variables $X_1, X_2, \dots$ from a finite alphabet $\mathcal{X} = 0, 1, 2, \dots, m$. For any sequence $x_1, x_2, \dots$, from this alphabet, we can place 0. in front of the sequence and consider it as a real number (base $m + 1$) between 0 and 1. Let X be the real-valued random variable $X = 0.X_1X_2\dots$. Then X has the following distribution function:

$$F_X(x) = \Pr\{X \leq x = 0.x_1x_2\dots\} \quad (13.53)$$

$$= \Pr\{0.X_1X_2\dots \leq 0.x_1x_2\dots\} \quad (13.54)$$

$$= \Pr\{X_1 < x_1\} + \Pr\{X_1 = x_1, X_2 < x_2\} + \dots \quad (13.55)$$

Now let $U = F_X(X) = F_X(0.X_1X_2\dots) = 0.F_1F_2\dots$. If the distribution on infinite sequences X^∞ has no atoms, then, by the lemma above, U has a uniform distribution on $[0, 1]$, and therefore the bits $F_1F_2\dots$ in the binary expansion of U are Bernoulli($\frac{1}{2}$) (i.e., they are independent and uniformly distributed on $\{0, 1\}$). These bits are therefore incompressible, and form a compressed representation of the sequence $0.X_1X_2\dots$. For Bernoulli or Markov models, it is easy to calculate the cumulative distribution function, as illustrated in the following example.

Example 13.3.1 Let $X_1, X_2, \dots, X_n$ be Bernoulli(p). Then the sequence $x^n = 110101$ maps into

$$\begin{aligned}
F(x^n) &= \Pr(X_1 < 1) + \Pr(X_1 = 1, X_2 < 1) \\
&\quad + \Pr(X_1 = 1, X_2 = 1, X_3 < 0) \\
&\quad + \Pr(X_1 = 1, X_2 = 1, X_3 = 0, X_4 < 1) \\
&\quad + \Pr(X_1 = 1, X_2 = 1, X_3 = 0, X_4 = 1, X_5 < 0) \\
&\quad + \Pr(X_1 = 1, X_2 = 1, X_3 = 0, X_4 = 1, X_5 = 0, X_6 < 1) \\
&\hspace{15em} (13.56)
\end{aligned}$$

$$= q + pq + p^2 \cdot 0 + p^2 q \cdot q + p^2 qp \cdot 0 + p^2 qpqq \quad (13.57)$$

$$= q + pq + p^2 q^2 + p^3 q^3. \quad (13.58)$$

Note that each term is easily computed from the previous terms. In general, for an arbitrary binary process $\{X_i\}$,

$$F(x^n) = \sum_{k=1}^n p(x^{k-1}0)x_k. \quad (13.59)$$

The probability transform thus forms an invertible mapping from infinite source sequences to incompressible infinite binary sequences. We now consider the compression achieved by this transformation on finite sequences. Let $X_1, X_2, \dots, X_n$ be a sequence of binary random variables of length n , and let $x_1, x_2, \dots, x_n$ be a particular outcome. We can treat this sequence as representing an interval $[0.x_1x_2 \dots x_n 000 \dots, 0.x_1x_2 \dots x_n 1111 \dots)$, or equivalently, $[0.x_1x_2 \dots x_n, 0.x_1x_2 \dots x_n + (\frac{1}{2})^n)$. This is the set of infinite sequences that start with $0.x_1x_2 \dots x_n$. Under the probability transform, this interval gets mapped into another interval, $[F_Y(0.x_1x_2 \dots x_n), F_Y(0.x_1x_2 \dots x_n + (\frac{1}{2})^n))$, whose length is equal to $P_X(x_1, x_2, \dots, x_n)$, the sum of the probabilities of all infinite sequences that start with $0.x_1x_2 \dots x_n$. Under the probability inverse transform, any real number u within this interval maps into a sequence that starts with $x_1, x_2, \dots, x_n$, and therefore given u and n , we can reconstruct $x_1, x_2, \dots, x_n$. The Shannon–Fano–Elias coding scheme described earlier allows one to construct a prefix-free code of length $\log \frac{1}{p(x_1, x_2, \dots, x_n)} + 2$ bits, and therefore it is possible to encode the sequence $x_1, x_2, \dots, x_n$ with this length. Note that $\log \frac{1}{p(x_1, \dots, x_n)}$ is the ideal code-word length for x^n .

The process of encoding the sequence with the cumulative distribution function described above assumes arbitrary accuracy for the computation. In practice, though, we have to implement all numbers with finite precision, and we describe such an implementation. The key is to consider

not infinite-precision points for the cumulative distribution function but intervals in the unit interval. Any finite-length sequence of symbols can be said to correspond to a subinterval of the unit interval. The objective of the arithmetic coding algorithm is to represent a sequence of random variables by a subinterval in $[0, 1]$. As the algorithm observes more input symbols, the length of the subinterval corresponding to the input sequence decreases. As the top end of the interval and the bottom end of the interval get closer, they begin to agree in the first few bits. These will be first few bits of the output sequence. As soon as the two ends of the interval agree, we can output the corresponding bits. We can therefore shift these bits out of the calculation and effectively scale the remaining intervals so that entire calculation can be done with finite precision. We will not go into the details here—there is a very good description of the algorithm and performance considerations in Bell et al. [41]

Example 13.3.2 (*Arithmetic coding for a ternary input alphabet*) Consider a random variable X with a ternary alphabet $\{A, B, C\}$, which are assumed to have probabilities 0.4, 0.4, and 0.2, respectively. Let the sequence to be encoded by ACAA. Thus, $F_l(\cdot) = (0, 0.4, 0.8)$ and $F_h(\cdot) = (0.4, 0.8, 1.0)$. Initially, the input sequence is empty, and the corresponding interval is $[0, 1)$. The cumulative distribution function after the first input symbol is shown in Figure 13.2. It is easy to calculate that the interval in the algorithm without scaling after the first symbol A is

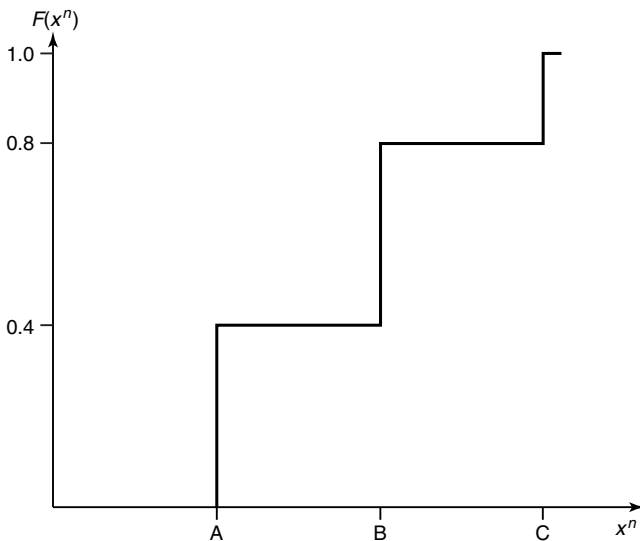


FIGURE 13.2. Cumulative distribution function after the first symbol.

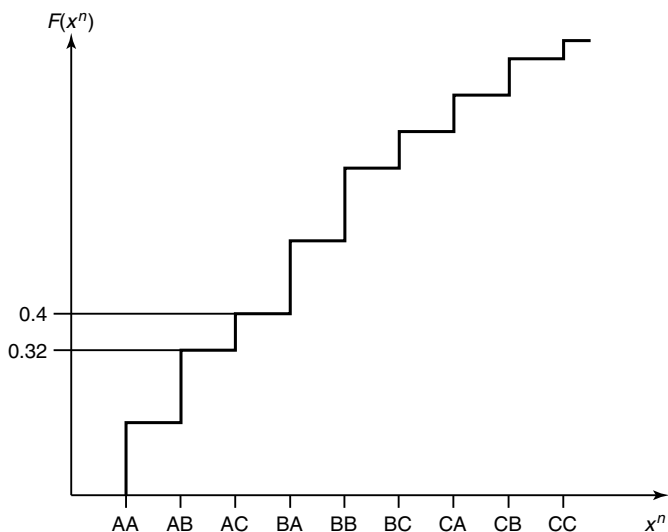


FIGURE 13.3. Cumulative distribution function after the second symbol.

$[0, 0.4)$; after the second symbol, C, it is $[0.32, 0.4)$ (Figure 13.3); after the third symbol A, it is $[0.32, 0.352)$; and after the fourth symbol A, it is $[0.32, 0.3328)$. Since the probability of this sequence is 0.0128, we will use $\log(1/0.0128) + 2$ (i.e., 9 bits) to encode the midpoint of the interval sequence using Shannon–Fano–Elias coding (0.3264, which is 0.010100111 binary).

In summary, the arithmetic coding procedure, given any length n and probability mass function $q(x_1x_2 \cdots x_n)$, enables one to encode the sequence $x_1x_2 \cdots x_n$ in a code of length $\log \frac{1}{q(x_1x_2 \cdots x_n)} + 2$ bits. If the source is i.i.d. and the assumed distribution q is equal to the true distribution p of the data, this procedure achieves an average length for the block that is within 2 bits of the entropy. Although this is not necessarily optimal for any fixed block length (a Huffman code designed for the distribution could have a lower average codeword length), the procedure is incremental and can be used for any blocklength.

13.4 LEMPEL–ZIV CODING

In Section 13.3 we discussed the basic ideas of arithmetic coding and mentioned some results on worst-case redundancy for coding a sequence from an unknown distribution. We now discuss a popular class of techniques for source coding that are universally optimal (their asymptotic